

CONNECT MARATHA — PRODUCT REQUIREMENTS DOCUMENT (PRD)

****Product Name:** Connect Maratha**

****Platform:** Native Flutter Android + iOS**

****Current Development Phase:** Flutter UI + Feature Implementation with Local/Mock Data**

****Future Backend:** Node.js + TypeScript/NestJS + PostgreSQL**

****State Management:** Riverpod**

****Future API Client:** Dio**

****Product Type:** Community + Professional Networking + Business + Services Platform**

****PRD Version:** 1.0**

1. PRODUCT VISION

Vision

Build a trusted digital ecosystem where community members can:

- Discover people
- Connect with the community
- Build professional relationships
- Discover businesses and services
- Participate in community activities
- Communicate with other members
- Discover jobs and opportunities
- Refer new members
- Participate in approved referral/earning programs in the future

Core Product Idea

DISCOVER



CONNECT



COMMUNICATE



COLLABORATE



GROW



STRENGTHEN THE COMMUNITY

2. PRODUCT PRINCIPLES

2.1 Existing Website Is the Source of Truth

The existing Connect Maratha website is the primary reference for:

- Existing features
- Existing functionality
- Content
- User flows
- Navigation
- Information architecture
- Terminology

The Flutter application should preserve these capabilities.

2.2 Native Mobile Experience

The Flutter application must be a TRUE native Flutter application.

DO NOT use:

- WebView
- HTML-to-Flutter conversion
- Screenshot-based UI
- Full-screen static images as UI
- Desktop web layouts copied directly into Flutter

Instead, convert the existing website functionality into proper mobile UX using native Flutter widgets.

2.3 Feature Parity

The Flutter application should eventually provide the major user-facing functionality available in the existing Connect Maratha website.

2.4 Backend Independence During Current Phase

For the CURRENT development phase:

Flutter UI



Riverpod



Repository



Local/Mock Data

Do NOT connect the Flutter application to the existing Node.js backend yet.

Future architecture:

Flutter UI



Riverpod



Repository



Dio



Node.js API



PostgreSQL

The UI must be designed so that backend integration can be added later without rewriting the screens.

2.5 Server Authority for Sensitive Features

When backend integration is implemented, the server must be authoritative for:

- Authentication
- Authorization
- User roles
- Referral attribution
- Referral qualification
- Wallet/earning calculations
- Moderation
- Financial operations
- Sensitive business rules

3. EXISTING PROJECT REFERENCE

The current website project is:

E:\new project

or, if the active/current project is the newer React/Vite project:

cm-web

The existing project contains:

- React/Vite frontend
- Node.js/Express backend
- Existing website pages
- Existing assets
- Existing functionality
- Existing data structures

IMPORTANT:

DO NOT MODIFY THE EXISTING WEBSITE PROJECT WHILE BUILDING THE FLUTTER APPLICATION.

The website should be treated as the functional/content reference.

The Flutter project is the actual mobile development project.

4. TARGET USERS

4.1 Regular Community Member

Primary user of the mobile application.

Capabilities:

- Create profile
- Manage profile
- Discover members
- Search members
- Connect with people
- Join groups
- Participate in community feed
- View events
- Discover businesses
- Discover services
- View heritage content
- Send messages
- Receive notifications
- Generate/view personal QR
- Refer new members

4.2 Business / Professional

Future capabilities:

- Business profile
- Professional profile

- Services
- Business discovery
- Leads/enquiries
- Networking
- Referrals

4.3 Community / Group Admin

Future capabilities:

- Manage approved groups
- Manage group members
- Moderate group content
- Manage group activities

4.4 Support / Customer Care

Future capabilities:

- User support
- Complaints
- Support tickets

4.5 Moderator

Future capabilities:

- Review reports
- Moderate content
- Moderate users
- Enforce community rules

4.6 Super Admin

Future capabilities:

- Full platform administration
- Roles
- Permissions
- Users
- Content
- Reports
- Analytics
- Referrals
- Configuration
- Audit logs

5. MAJOR PRODUCT MODULES

The Connect Maratha application should contain the following major areas:

1. Authentication
2. Home
3. Profile
4. Community
5. Connections
6. Business
7. Events
8. Heritage
9. Services

- 10. Jobs
- 11. Messaging
- 12. Referrals
- 13. Notifications
- 14. Settings
- 15. Future Wallet/Earnings
- 16. Future Administration

6. AUTHENTICATION

Objective

Allow users to securely access their Connect Maratha account.

Existing Functionality to Preserve

- Login
- Registration
- Mobile/member identification
- Password/OTP concept
- Role selection where applicable
- Forgot password
- Session
- Member identity

Flutter Screens

Splash



Login

└─ Register

└─ Forgot Password



Home

Current Phase

Use mock/local authentication.

Example:

- Demo user
- Demo member
- Demo role

DO NOT connect authentication to Node.js yet.

Future Backend

Node.js will eventually handle:

- Authentication
- OTP/password
- JWT access tokens
- Refresh tokens
- Session/device management
- Logout
- Account recovery
- Rate limiting
- Brute-force protection

- Account deletion

7. SPLASH SCREEN

Purpose

Introduce the Connect Maratha brand before entering the application.

Design Direction

- Clean
- Premium
- Minimal
- Strong brand identity
- Rajmudra/Maratha heritage reference
- Short animation
- White/cream or approved brand background

The splash must not become an overloaded promotional screen.

8. HOME / DASHBOARD

Objective

The Home screen is the primary discovery dashboard.

Home Should Provide

- User greeting
- Profile/avatar
- Member card
- Personal QR access
- Search
- Notifications
- Quick actions
- Community activity
- Upcoming events
- Business discovery
- Service discovery
- Heritage highlights
- Recent activity

UX Principle

Do not put every feature on the Home screen.

Home should act as a discovery and shortcut dashboard.

Visual Direction

The approved visual direction may use:

- Deep black/AMOLED areas where appropriate
- Saffron/gold accents
- Premium cards
- Heritage imagery
- Subtle borders
- Strong typography

- Modern mobile layout

Do not use one giant image as the entire UI.

9. समुदाय — COMMUNITY

Purpose

The Community section is one of the core features of Connect Maratha.

Core concept:

"मराठा बांधवांशी जोडा, समाज सशक्त करा"

Meaning:

Connect with community members and strengthen the community.

Community Components

9.1 Member Discovery

Users should be able to discover members using:

- Name
- City
- Profession
- Business

- Skills
- Interests
- Location

9.2 Location Filters

Existing website concepts include:

- सर्व
- पुणे Connect
- मुंबई Connect
- नाशिक Connect
- महाराष्ट्राबाहेरील

These concepts should be adapted into mobile-friendly filters.

9.3 Member Cards

Member cards can contain:

- Profile photo
- Name
- Member ID
- Profession
- City
- Skills/interests
- Connection status

9.4 Member Profile

Tapping a member should open the member profile.

9.5 Community Groups

Users should be able to:

- Discover groups
- View group details
- Join groups
- View group members
- Participate in group activity

9.6 Community Feed

Users should eventually be able to:

- Create posts
- View posts
- Like/react
- Comment
- Share
- Report

9.7 Direct Messaging

Users should eventually be able to message other members.

10. MEMBER PROFILE

Profile Information

A member profile can contain:

- Profile photo
- Name
- Member ID
- Location
- Profession
- Business
- Skills
- Interests
- About
- Membership information
- Community activity
- QR
- Referral information

Privacy

Future backend should support profile visibility controls.

11. PERSONAL QR / DIGITAL MEMBER CARD

Objective

Give every member a digital identity card and QR.

Member QR

A member can open:

"माझा QR"

or:

"My QR"

Example:

CONNECT मराठा

[Profile Photo]

Abhay Gond

CM102483

Software Developer

Pune

[QR CODE]

Scan to Connect

QR Use Cases

Use Case 1 — Member Discovery

Existing member scans another member's QR.

Flow:

User A



My QR



User B scans



Member Profile



Connect / Message

Use Case 2 — Referral

Existing member can invite a new person.

Flow:

Existing Member



Refer Someone



Generate Referral QR



New Person Scans



Connect Maratha Join Screen



Registration



Referral Recorded

Security Requirement

DO NOT encode private user information directly into the QR.

Do not put:

- Phone number
- Email
- Password
- Private information

inside the QR.

Use a secure/non-guessable identifier or referral token.

Example:

<https://connectmaratha.com/join?ref=8FJ72KQ>

Current Phase

Generate mock/local QR data.

Future Phase

Node.js backend will:

- Generate referral identifier
- Validate referral
- Identify referrer

- Record attribution
- Prevent duplicate/fraudulent attribution

12. CONNECTIONS

Future Features

- Send connection request
- Accept
- Reject
- Remove connection
- Mutual connections
- Suggested connections
- Follow/follower relationship where applicable

The UI should be designed now so these flows can be connected to the backend later.

13. BUSINESS NETWORK

Objective

The Business section should be a structured professional/business showcase, not simply a social feed.

Business Categories

Potential profiles include:

- Business owners
- Entrepreneurs
- Professionals
- Creators
- Social workers
- Organizations
- Community contributors

Features

- Business directory
- Business profile
- Business category
- Description
- Location
- Website/contact options
- Products/services
- Professional profile
- Skills
- Experience
- Achievements
- Search
- Filters
- Networking
- Introductions
- Referrals
- Enquiries

Flutter Screens

Business

- └─ Directory
- └─ Search
- └─ Filters
- └─ Business Profile
- └─ Professional Profile
- └─ Enquiry

14. SERVICES

Objective

Help users discover businesses, professionals and service providers.

Features

- Service categories
- Service listings
- Provider profiles
- Search
- Location
- Enquiry
- Request service

Current Phase

Use mock/local service data.

15. EVENTS / उपक्रम

Features

- Upcoming events
- Event details
- Date
- Time
- Location
- Description
- Organizer
- Registration
- Announcements

Current Phase

Use local/mock event data.

Future Phase

Connect to Node.js event APIs.

16. HERITAGE & CULTURE

Heritage is an important Connect Maratha content area.

Main Sections

Heritage

- └─ History
- └─ Forts
- └─ Warriors
- └─ Important Dates
- └─ Culture
- └─ Knowledge
- └─ Heritage Content

History

Provide mobile-friendly historical content.

Forts

Users should be able to:

- Browse forts
- Search forts
- Filter forts
- View fort cards
- Open fort details
- View images
- Read history
- View statistics
- View attractions
- View trek/travel information

Warriors

Users should be able to:

- Browse warriors
- View warrior profile/content
- Read historical information
- View related content

Important Dates

Provide a mobile-friendly historical timeline/calendar experience.

17. JOBS & OPPORTUNITIES

Future Features

- Job listings
- Job search
- Filters
- Job details
- Employer information
- Application
- Application status

Current Phase

Use mock/local job data if implementing the UI.

18. MESSAGING

Future Features

One-to-One Chat

- Text messages
- Media
- Read status
- Delivery status
- Typing indicator
- Presence

Group Chat

- Group messages
- Members
- Media
- Read status

Safety

- Block
- Report

Current Phase

Create the UI and mock conversations.

Do not implement real WebSocket/backend communication yet.

Future Architecture

Flutter



Dio / REST



Node.js



Socket.IO/WebSocket



PostgreSQL/Redis

19. NOTIFICATIONS

Notification Categories

- Messages
- Connections
- Referrals
- Business enquiries
- Community
- Jobs
- Services
- System

Current Phase

Use mock notification data.

Future Phase

Use backend push notifications and FCM.

20. REFERRAL SYSTEM

Objective

Allow existing members to introduce new people to Connect Maratha.

Basic Flow

Member A



Referral QR / Link



New User



Registration



Member A becomes Referrer



Referral recorded

Referral Data

Future backend should track:

- Referrer
- Referred user

- Referral code
- Source
- Timestamp
- Status
- Qualification
- Campaign where applicable

Important

Referral attribution must be server-controlled.

Never trust:

- Client-supplied referral credit
- Client-supplied commission
- Client-supplied wallet balance

Current Phase

Build:

- Referral UI
- QR UI
- Referral history UI
- Mock referral data

Do not implement real monetary calculations yet.

21. WALLET / EARNINGS

This is a future module.

Potential features:

- Wallet balance
- Pending earnings
- Approved earnings
- Transactions
- Reversals
- Withdrawal
- Settlement status

Important

Wallet/earning calculations must never be controlled by Flutter.

They must be calculated and validated server-side.

22. PROFILE & SETTINGS

Profile

- View profile
- Edit profile
- Profile photo
- Personal information
- Professional information
- Membership

- QR
- Referrals
- Activity

Settings

- Privacy
- Notifications
- Security
- Sessions/devices
- Blocked users
- Account deletion

23. NAVIGATION

The application should use a clear mobile navigation system.

Possible primary navigation:

Home

Community

Business

Profile

Additional features should be accessible through:

- Home shortcuts
- Community
- Search

- Profile
- Secondary navigation
- Contextual navigation

Do not overcrowd the bottom navigation.

Use `go_router`.

Do not introduce another navigation framework.

24. UI/UX DESIGN REQUIREMENTS

Design Personality

The app should feel:

- Premium
- Modern
- Trustworthy
- Cultural
- Community-oriented
- Professional
- Mobile-first

Visual Language

Use:

- White/cream

- Saffron
- Gold
- Deep charcoal/black
- Strong typography
- Subtle cultural motifs
- High-quality heritage imagery

Cultural elements should support the product and must not make functional screens difficult to use.

Avoid

- Excessive gradients
- Excessive gold
- Excessive shadows
- Excessive rounded cards
- Tiny text
- Overcrowded layouts
- Random colors
- Generic default Flutter UI
- Desktop website copied directly into mobile
- Giant image-based screens

25. RESPONSIVE DESIGN

The app must work on:

- Small Android phones
- Large Android phones
- iPhones

- Different screen aspect ratios

Do not hard-code screen dimensions.

Avoid code such as:

width: 390

height: 844

Use:

- MediaQuery
- LayoutBuilder
- Flexible
- Expanded
- SafeArea
- Slivers
- Responsive constraints

26. ACCESSIBILITY

Consider:

- Readable font sizes
- Strong color contrast
- Semantic labels
- Touch target sizes
- Screen reader compatibility
- Dynamic text

- Do not use color as the only status indicator

27. UI STATES

Every major screen should support:

1. Loading
2. Success
3. Empty
4. Error
5. Offline
6. Unauthorized/session expired where applicable

Example:

If no events exist:

Show a meaningful empty state.

Do not show a blank screen.

28. ANIMATION

Use subtle animations for:

- Page transitions
- Button feedback

- Tab transitions
- Card appearance
- Favorite actions
- QR actions
- Bottom navigation transitions

Animations must improve UX.

Avoid excessive decorative animation.

29. FLUTTER ARCHITECTURE

Recommended structure:

lib/

core/

theme/

routing/

widgets/

constants/

utils/

data/

models/

repositories/

datasources/

features/

auth/

home/
community/
connections/
business/
events/
heritage/
services/
jobs/
messaging/
referrals/
profile/
notifications/

main.dart

Architecture

UI



Riverpod



Repository



Local/Mock Data Source

Future:

UI



Riverpod



Repository



Dio



Node.js API



PostgreSQL

30. STATE MANAGEMENT

Use Riverpod.

Do NOT introduce:

- Provider
- Bloc
- GetX
- MobX
- Another state management framework

Use Riverpod for:

- Authentication state
- Current user
- Home data
- Community data
- Business data
- Events
- Heritage

- Profile
- Referrals
- Notifications

Keep simple UI-only state local when appropriate.

Examples:

- Password visibility
- Temporary tab selection
- Animation state
- Temporary form state

31. CURRENT DATA ARCHITECTURE

For the current development phase:

Flutter UI



Riverpod



Repository



Mock/Local Data

Possible repositories:

- MockAuthRepository
- MockUserRepository

- MockCommunityRepository
- MockBusinessRepository
- MockEventsRepository
- MockHeritageRepository
- MockReferralRepository
- MockNotificationRepository

The UI must not directly manipulate mock data.

32. FUTURE BACKEND ARCHITECTURE

When backend integration begins:

Flutter



Riverpod



Repository



Dio



Node.js + TypeScript/NestJS



PostgreSQL

Future production stack may include:

- Node.js
- TypeScript

- NestJS
- REST API
- WebSocket/Socket.IO
- PostgreSQL
- Prisma
- Redis
- BullMQ
- Firebase Cloud Messaging
- Object storage
- Docker
- CI/CD
- Monitoring

33. CURRENT DEVELOPMENT SCOPE

Phase 1 — Foundation

Implement:

- Flutter project structure
- Theme
- Routing
- Riverpod
- Splash
- Login
- Registration
- Basic local authentication state

Phase 2 — Core Member Experience

Implement:

- Home
- Profile
- Edit Profile
- Member QR
- Community
- Member discovery
- Member profile
- Groups

Phase 3 — Content

Implement:

- History
- Forts
- Warriors
- Important Dates
- Heritage
- Events

Phase 4 — Networking

Implement:

- Business
- Business profile
- Services
- Connections

- Referrals

Phase 5 — Communication

Implement:

- Community feed
- Messaging UI
- Notifications UI

Phase 6 — Backend Integration

Connect:

Flutter



Node.js API



PostgreSQL

Phase 7 — Production

Implement:

- Production authentication
- Security
- Push notifications
- Real-time communication
- Media storage
- Backend validation
- Testing

- Monitoring
- Deployment

34. OUT OF SCOPE FOR CURRENT PHASE

DO NOT implement actual backend integration yet.

Do NOT:

- Connect Node.js APIs
- Connect PostgreSQL
- Connect production authentication
- Connect WebSockets
- Connect FCM
- Implement real wallet calculations
- Implement real referral earnings
- Implement real payment processing
- Connect production cloud storage
- Add production secrets

Only prepare the architecture for future integration.

35. EXISTING WEBSITE → FLUTTER RULE

For every feature:

1. Inspect the existing website.

2. Understand what it does.
3. Identify the user flow.
4. Identify the data displayed.
5. Identify actions.
6. Identify related pages.
7. Convert the desktop UX into mobile UX.
8. Implement the Flutter screen.
9. Use local/mock data.
10. Preserve the feature behavior.
11. Test the screen.
12. Do not modify the existing website.

The website is the functional source of truth.

The Flutter design is the mobile-native implementation.

36. QUALITY REQUIREMENTS

Before considering a screen complete, verify:

- Correct functionality
- Correct content
- Correct navigation
- Responsive layout
- No overflow
- Keyboard-safe forms
- Proper loading state
- Proper empty state
- Proper error state

- Proper spacing
- Proper typography
- Good accessibility
- Good touch targets
- Smooth scrolling
- Reasonable performance
- Consistent design system

37. TESTING

Run:

`dart format .`

`flutter analyze`

`flutter test`

`flutter build apk --debug`

Test on the available Android device.

Also test:

- Navigation
- Back navigation
- Forms
- Keyboard behavior
- Scrolling

- Empty states
- Error states
- Different screen sizes

38. SUCCESS CRITERIA

The current Flutter phase is successful when:

Feature Parity

Major user-facing functionality of the existing Connect Maratha website has corresponding Flutter screens/flows.

Native Experience

The application feels like a native mobile application rather than a website inside a phone.

Architecture

The UI does not directly depend on backend implementation.

State Management

Riverpod manages application-level state.

Data Layer

Mock/local repositories can later be replaced with API repositories.

Code Quality

Flutter analysis and tests pass.

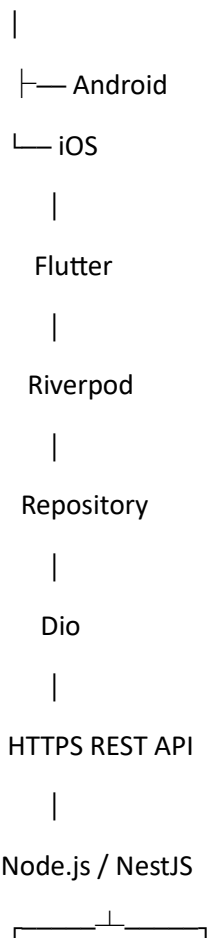
Build

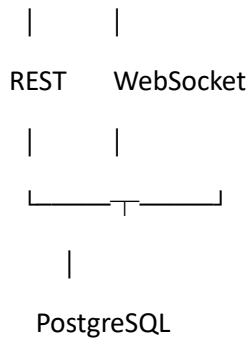
Debug APK builds successfully.

39. FINAL PRODUCT ARCHITECTURE

Final intended architecture:

CONNECT MARATHA





Supporting future infrastructure:

- Redis
- FCM
- Object Storage
- Background Jobs
- Admin Platform
- Monitoring
- CI/CD

40. PRODUCT DEVELOPMENT PRINCIPLE

The most important rule for this project is:

DO NOT BUILD A COMPLETELY DIFFERENT APP.

Build the mobile version of the existing Connect Maratha product.

Use:

EXISTING WEBSITE



Existing Features

+

Existing Content

+

Existing User Flows

↓

Native Mobile UX

↓

Flutter

↓

Riverpod

↓

Mock/Local Data

↓

Future Node.js API

↓

PostgreSQL

The goal is:

EXISTING CONNECT MARATHA FEATURES

+

NATIVE MOBILE EXPERIENCE

+

PREMIUM UI/UX

+

CLEAN ARCHITECTURE

+

FUTURE BACKEND READINESS